A PROJECT REPORT

ON

ArtVault

Online Art Gallery & Auction Platform

Submitted in partial fulfillment of the requirements for the award of degree/course completion

Submitted By

[Student Name]

[Roll Number / Enrollment Number]

Under the Guidance of

[Guide / Faculty Name]

[Department Name]

[College / University Name]

Academic Year 2025-2026

CERTIFICATE

This is to certify that the project report entitled "ArtVault - Online Art Gallery & Auction Platform" has been prepared and submitted by [Student Name] in partial fulfillment of the requirements of the academic project work. The project has been developed using the MERN stack and demonstrates a complete web-based platform for artwork discovery, artist uploads, real-time auctions, bidding, mock payments, notifications, and administrative management.

The work presented in this report is based on the project files, database models, frontend pages, backend APIs, and implementation available in the ArtVault source code.

Signature of Guide

[Guide / Faculty Name]

DECLARATION

I hereby declare that this project report titled "ArtVault - Online Art Gallery & Auction Platform" is prepared from the implemented project work. The contents describe the design, development, modules, technologies, database structure, testing, and screenshots of the application. The report has been prepared for academic submission and all project-specific information has been adapted from the actual ArtVault project.

Date: _____

Place: _____

Signature: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to my guide, teachers, and institution for providing the opportunity and support to complete this project. Their guidance helped in understanding the software development process, full-stack architecture, frontend-backend integration, database design, and testing methodology.

I also thank everyone who provided feedback during the development of ArtVault. The project helped strengthen practical knowledge of React, Node.js, Express, MongoDB, REST APIs, authentication, role-based authorization, file upload handling, and real-time communication using Socket.IO.

ABSTRACT

ArtVault is an Online Art Gallery & Auction Platform designed to connect artists, buyers, and administrators through a single digital marketplace. The platform allows artists to upload artwork, manage artwork details, create auctions, and track auction status. Buyers can browse artwork, participate in live auctions, place bids, manage orders, and view payment receipts. Administrators can monitor users, artworks, auctions, and platform statistics.

The project is developed using the MERN stack. The frontend uses React with Vite, Redux Toolkit, React Router, Axios, and Socket.IO client. The backend uses Node.js, Express, MongoDB with Mongoose, JWT-based authentication, Multer for artwork uploads, and Socket.IO for real-time auction updates. The system includes role-based access control for Artist, Buyer, and Admin roles.

The application demonstrates core concepts of modern full-stack development including REST API design, secure authentication, protected routes, reusable components, responsive UI design, database modeling, live bid updates, and structured dashboard workflows.

TABLE OF CONTENTS

1. Certificate
2. Declaration
3. Acknowledgement
4. Abstract
5. 1. Introduction
6. 2. System Analysis
7. 3. Requirement Specification
8. 4. System Design
9. 5. Module Description
10. 6. Implementation Details
11. 7. User Interface Screenshots
12. 8. Testing
13. 9. Future Scope
14. 10. Conclusion
15. 11. References

1. INTRODUCTION

The art market increasingly depends on digital platforms for visibility, sales, and audience engagement. Traditional galleries are limited by location, operating hours, and manual coordination between artists and buyers. ArtVault provides a digital solution where original artworks can be listed, discovered, auctioned, and purchased through an integrated web application.

The platform supports artwork browsing, category and price-based filtering, detailed artwork pages, live auctions, bid tracking, mock payment flow, notifications, and separate dashboards for artists, buyers, and administrators. It is designed as a full-stack MERN application with a responsive dark art-gallery aesthetic.

1.1 Project Title

ArtVault - Online Art Gallery & Auction Platform

1.2 Problem Statement

Artists need a simple way to display and sell their work online, while buyers need a trusted system to discover artwork and participate in transparent auctions. Manual auction handling can be slow, error-prone, and difficult to monitor. ArtVault solves this by providing a centralized platform with role-based workflows and real-time bid updates.

1.3 Objectives

- To provide an online gallery where users can browse artworks by category, price, status, and search keyword.
- To allow artists to upload artwork and manage artwork and auction records.
- To support live auctions with current bid tracking and countdown timers.
- To allow buyers to place bids, view bid history, complete mock payments, and view receipts.
- To provide notifications for auction, bid, and payment events.
- To provide an admin dashboard for user, artwork, auction, and platform overview management.

2. SYSTEM ANALYSIS

2.1 Existing System

In many traditional art sale workflows, artists display work through physical galleries, direct contacts, or static catalogues. Auction updates may be handled manually and buyers may not receive instant bid status information. Such systems can limit reach, reduce transparency, and require more coordination.

2.2 Proposed System

The proposed system is a web-based MERN application that digitizes artwork listing, auction participation, payment simulation, and user management. It provides a structured interface for three user roles: Artist, Buyer, and Admin. Artists upload and auction artwork, buyers browse and bid, and admins monitor the platform.

2.3 Advantages of Proposed System

- Centralized artwork discovery and auction management.
- Role-based authentication and protected dashboards.
- Real-time bid updates through Socket.IO.
- Responsive frontend with reusable React components.
- Structured database models for users, artworks, auctions, bids, orders, and notifications.
- Mock payment flow that can later be integrated with Razorpay, Stripe, or another payment gateway.

3. REQUIREMENT SPECIFICATION

3.1 Hardware Requirements

- Processor: Intel i3 / Ryzen 3 or higher
- RAM: 4 GB minimum, 8 GB recommended
- Storage: 500 MB for project files and dependencies
- Network: Internet connection for cloud image URLs and package installation

3.2 Software Requirements

- Operating System: Windows / Linux / macOS
- Frontend Runtime: Node.js 18+ with Vite
- Backend Runtime: Node.js with Express
- Database: MongoDB local or MongoDB Atlas
- Editor: Visual Studio Code
- Browser: Microsoft Edge / Chrome / Firefox

3.3 Technology Stack

- Frontend: React 19, Vite, Redux Toolkit, React Router, Axios, React Hot Toast, Socket.IO Client.
- Backend: Node.js, Express, CORS, dotenv, JWT, bcryptjs, Multer, Socket.IO.
- Database: MongoDB with Mongoose schemas.
- Development Tools: ESLint, Nodemon, npm scripts, Git.

4. SYSTEM DESIGN

ArtVault follows a client-server architecture. The React frontend communicates with the Express backend through REST APIs. MongoDB stores persistent records, while Socket.IO enables live auction bid events. Authentication uses JWT tokens stored on the client and sent through authorization headers for protected requests.

4.1 Data Flow

- A user registers or logs in through the authentication API.
- The backend validates credentials, generates a JWT token, and returns user role information.
- The frontend stores the user session and controls route access through protected routes.
- Artists upload artwork and create auctions through protected API endpoints.
- Buyers browse public artwork and auction data and place bids through authenticated bid APIs.
- Socket.IO broadcasts new bid updates to users viewing the same auction.
- Orders and notifications are generated for payment and auction-related events.

4.2 Database Design

- User: stores name, email, hashed password, role, avatar, bio, and active status.
- Artwork: stores artist reference, title, description, category, medium, dimensions, base price, image URL, tags, featured flag, and status.
- Auction: stores artwork reference, artist reference, starting bid, current highest bid, highest bidder, end date, status, and winner.
- Bid: stores auction reference, buyer reference, bid amount, and bid timestamp.
- Order: stores buyer, artwork, auction, amount, payment status, delivery status, transaction details, and shipping address.
- Notification: stores receiver, type, message, read status, and related event information.

5. MODULE DESCRIPTION

5.1 Authentication Module

The authentication module includes registration, login, current user retrieval, and profile update features. Passwords are hashed with bcryptjs, and JWT is used to authenticate protected requests. Role-based protected routes separate Artist, Buyer, and Admin workflows.

5.2 Gallery Module

The gallery module displays artworks with filters for category, status, sort order, search text, and price range. Each artwork card links to a detailed page showing title, artist, category, description, price, status, and auction information when applicable.

5.3 Artwork Upload Module

Artists can upload artwork with title, description, category, medium, dimensions, base price, tags, and artwork image. The backend uses Multer to handle file upload and stores artwork metadata in MongoDB.

5.4 Auction Module

Artists can create auctions for available artworks. Buyers can view active auctions, inspect details, track countdown timers, view current highest bids, and place bids. Socket.IO updates auction pages when new bids are placed.

5.5 Order and Payment Module

The order module supports mock payment after a buyer wins or purchases artwork. Paid orders include transaction information and receipt display. This module is designed so a real payment provider can be integrated later.

5.6 Notification Module

Notifications inform users about bids, auction wins, payment events, and other general updates. Users can mark notifications as read or mark all notifications as read.

5.7 Admin Module

The admin dashboard provides platform statistics and management views for users, artworks, and auctions. Admin users can monitor the system and toggle or delete user accounts where required.

6. IMPLEMENTATION DETAILS

6.1 Frontend Implementation

The frontend is organized into pages, components, Redux slices, and API services. React Router manages navigation. Redux Toolkit stores authentication, artworks, auctions, bids, orders, and notifications. Axios centralizes API calls and attaches JWT tokens to protected requests.

6.2 Backend Implementation

The backend is structured into routes, controllers, middleware, models, config, and utilities. Express handles API routing. Mongoose manages database schemas. Auth middleware protects routes by verifying JWT tokens. Role middleware restricts access to Artist, Buyer, and Admin operations.

6.3 API Endpoints

- /api/auth/register, /api/auth/login, /api/auth/me, /api/auth/profile
- /api/artworks, /api/artworks/my, /api/artworks/:id
- /api/auctions, /api/auctions/my, /api/auctions/:id, /api/auctions/:id/end
- /api/bids, /api/bids/auction/:auctionId, /api/bids/my
- /api/orders, /api/orders/my, /api/orders/:id, /api/orders/:id/pay
- /api/notifications, /api/notifications/:id/read, /api/notifications/read-all
- /api/admin/stats, /api/admin/users, /api/admin/artworks

7. USER INTERFACE SCREENSHOTS

The following screenshots were captured from the running local ArtVault application at http://localhost:5173.

Figure 7.1: Home Page

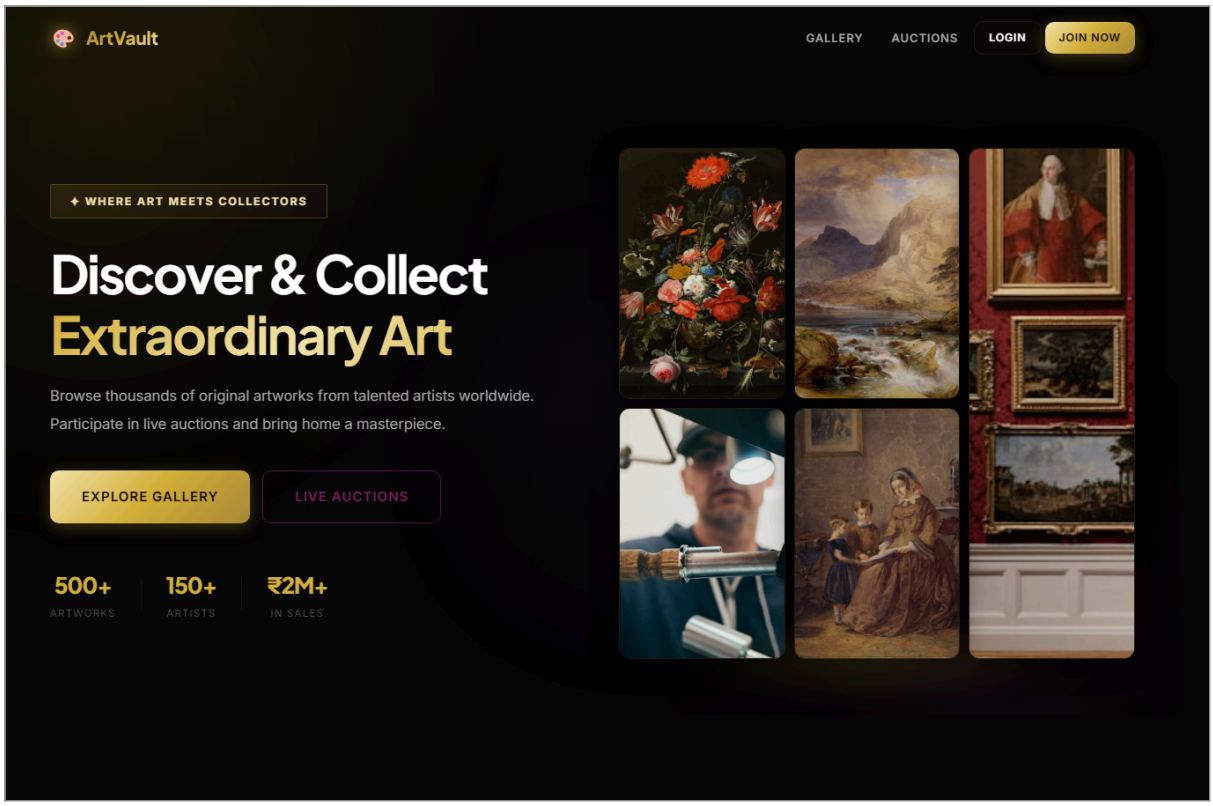


Figure 7.2: Artwork Gallery

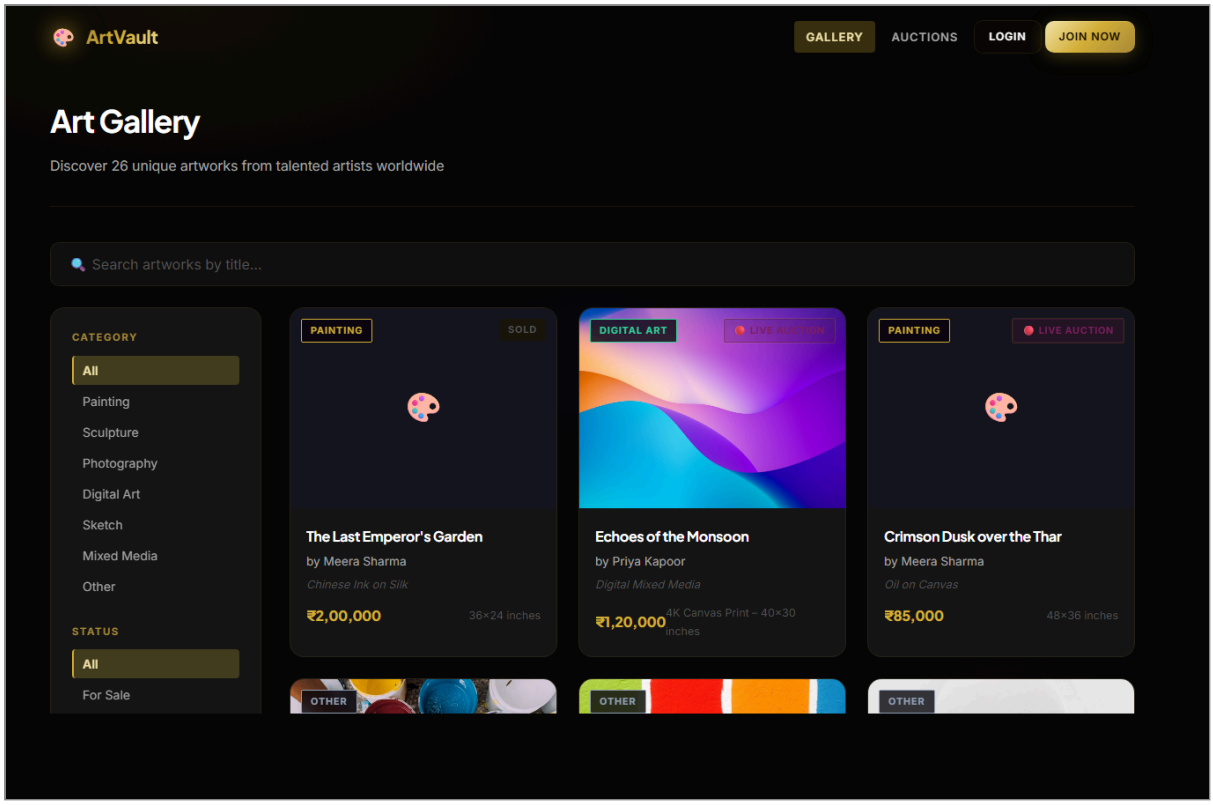


Figure 7.3: Auction Listing

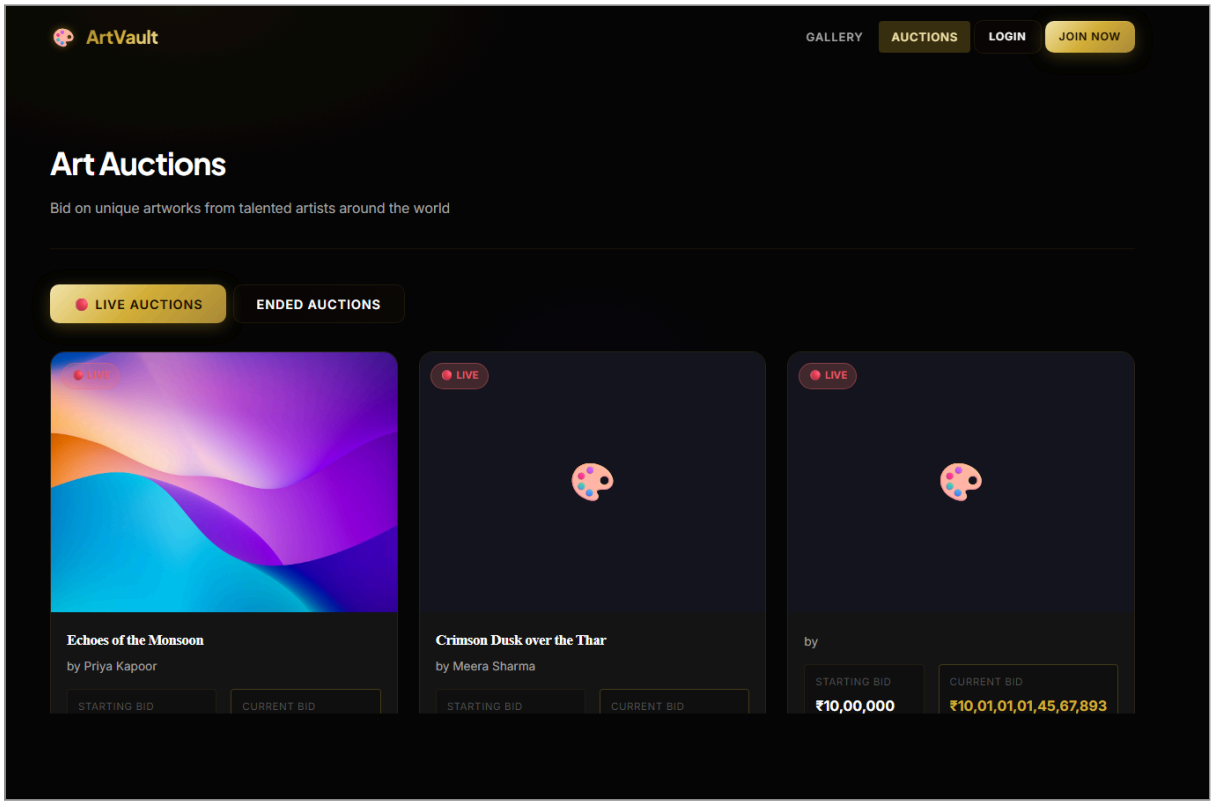


Figure 7.4: Artwork Details Page

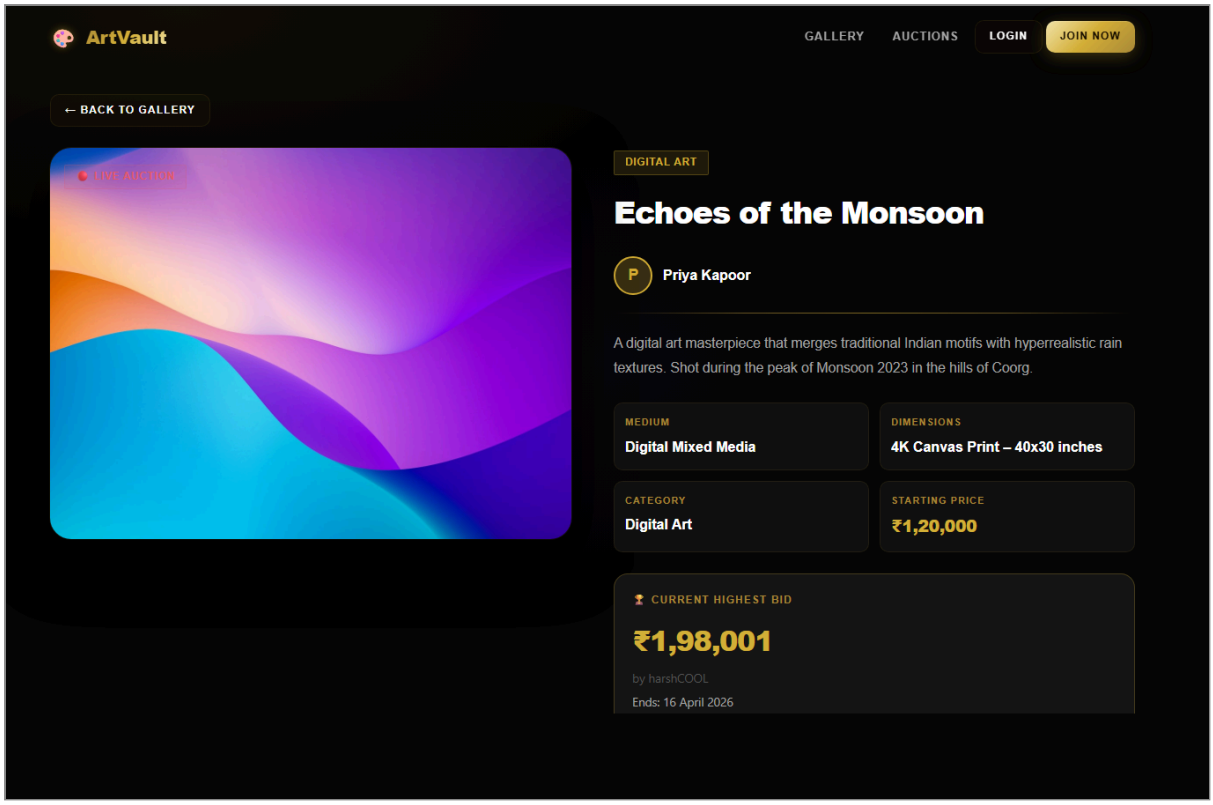


Figure 7.5: Auction Detail and Bid Panel

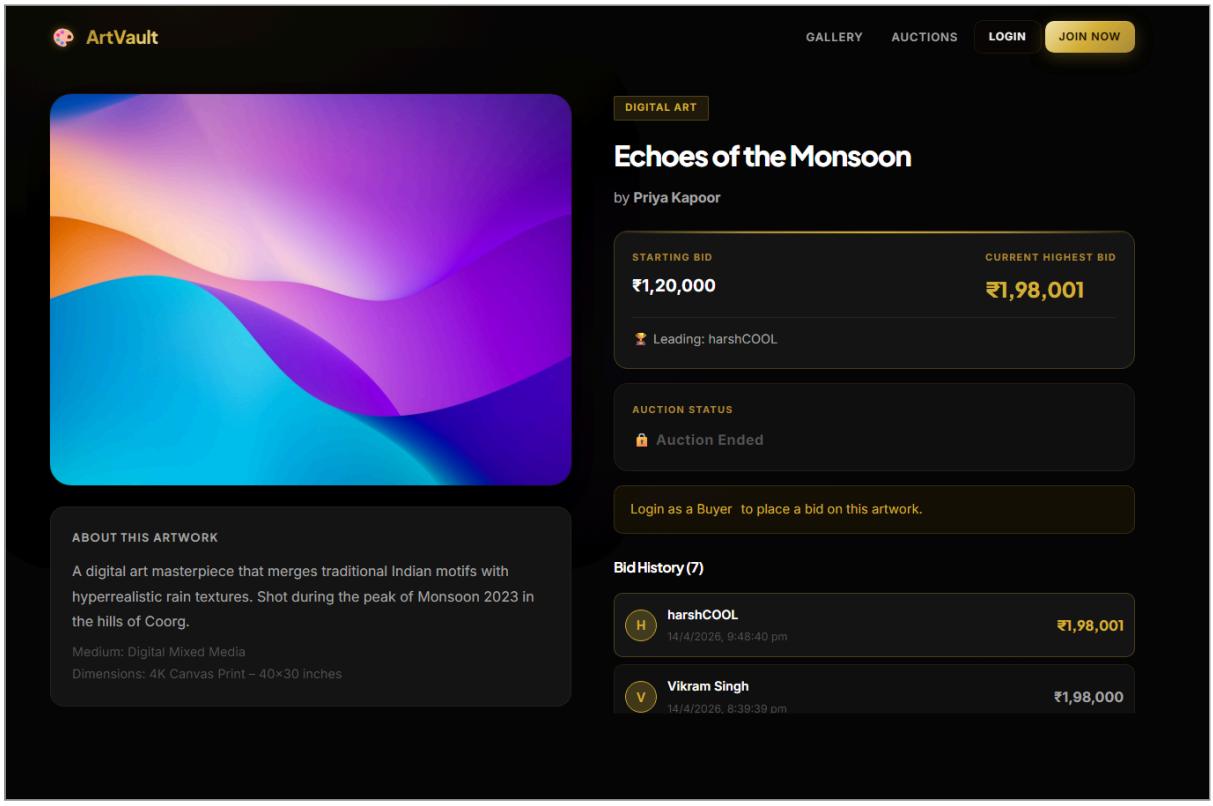


Figure 7.6: Login Page

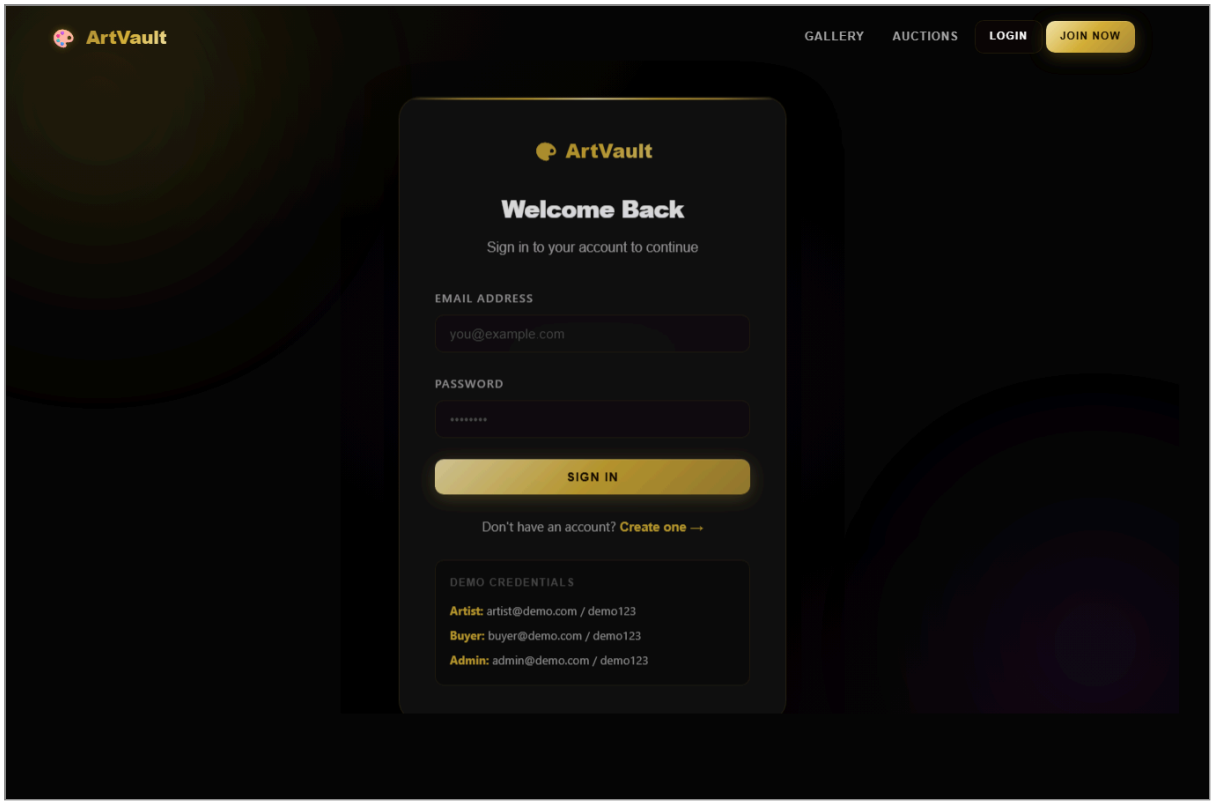


Figure 7.7: Registration Page

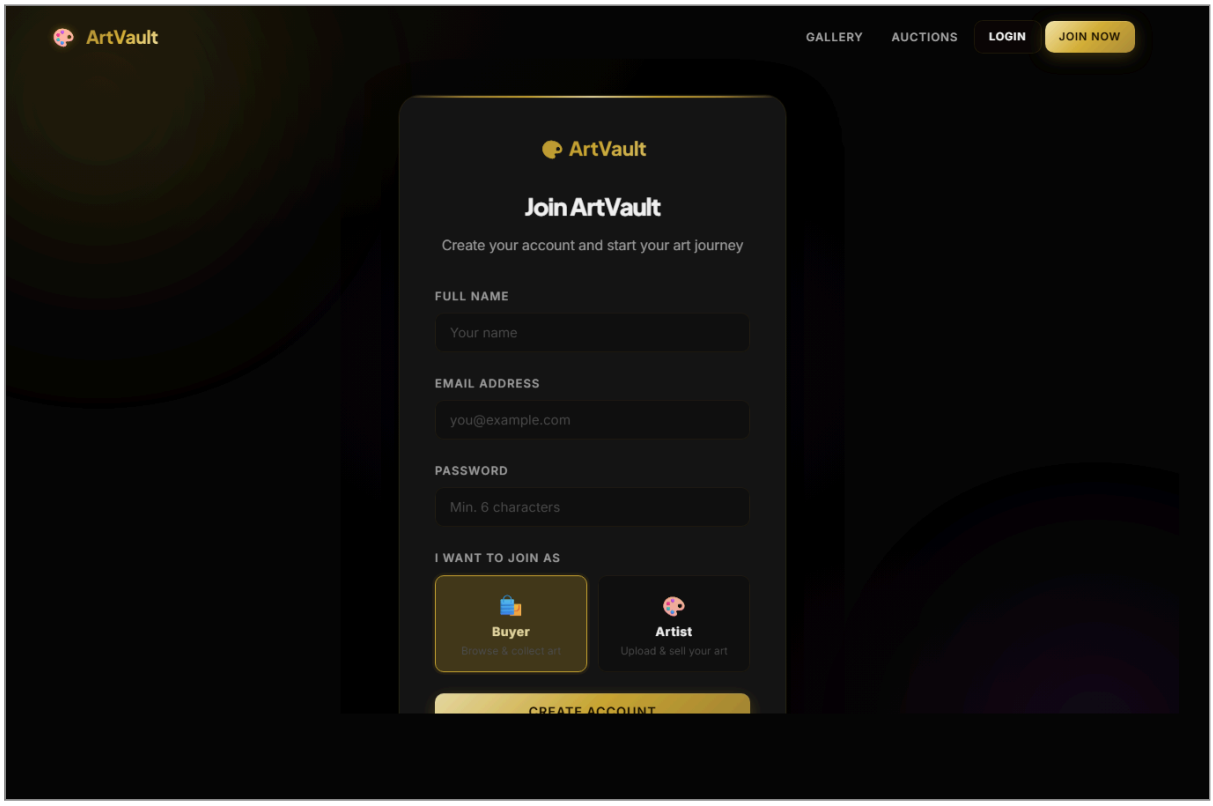


Figure 7.8: Protected Artist Upload Route

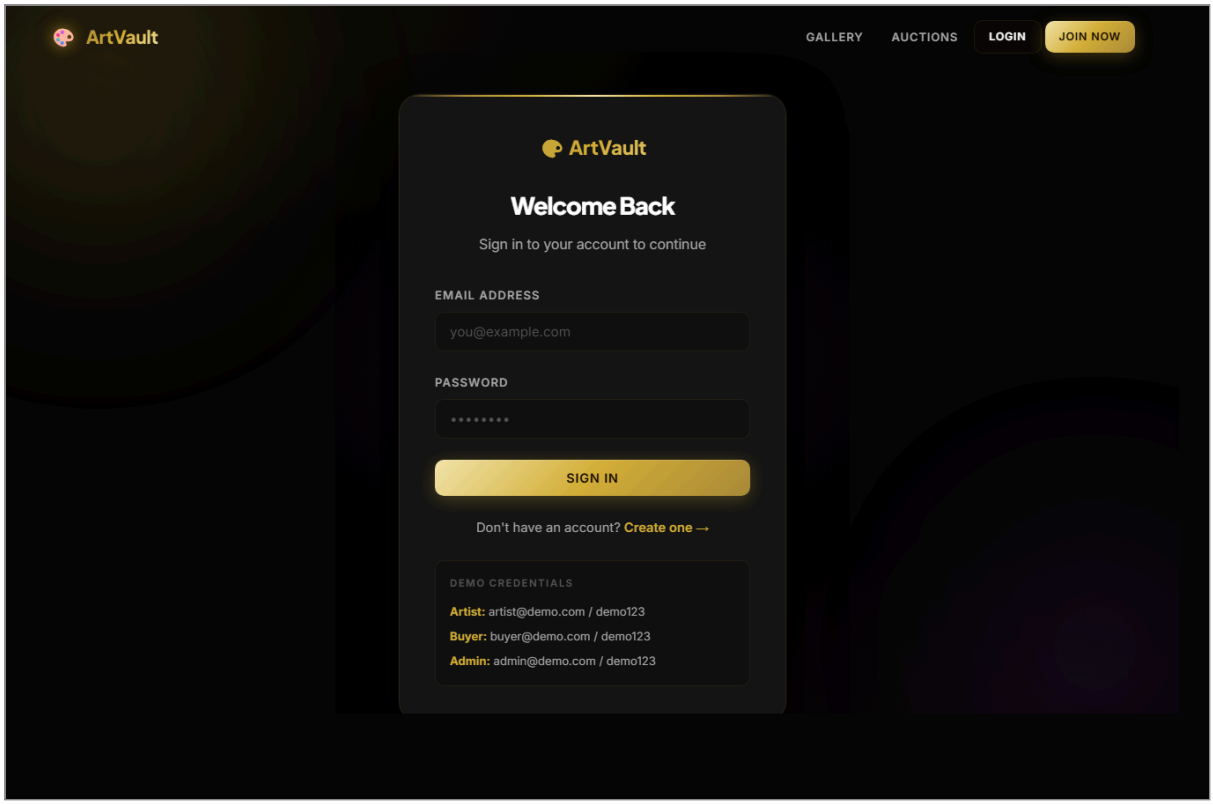
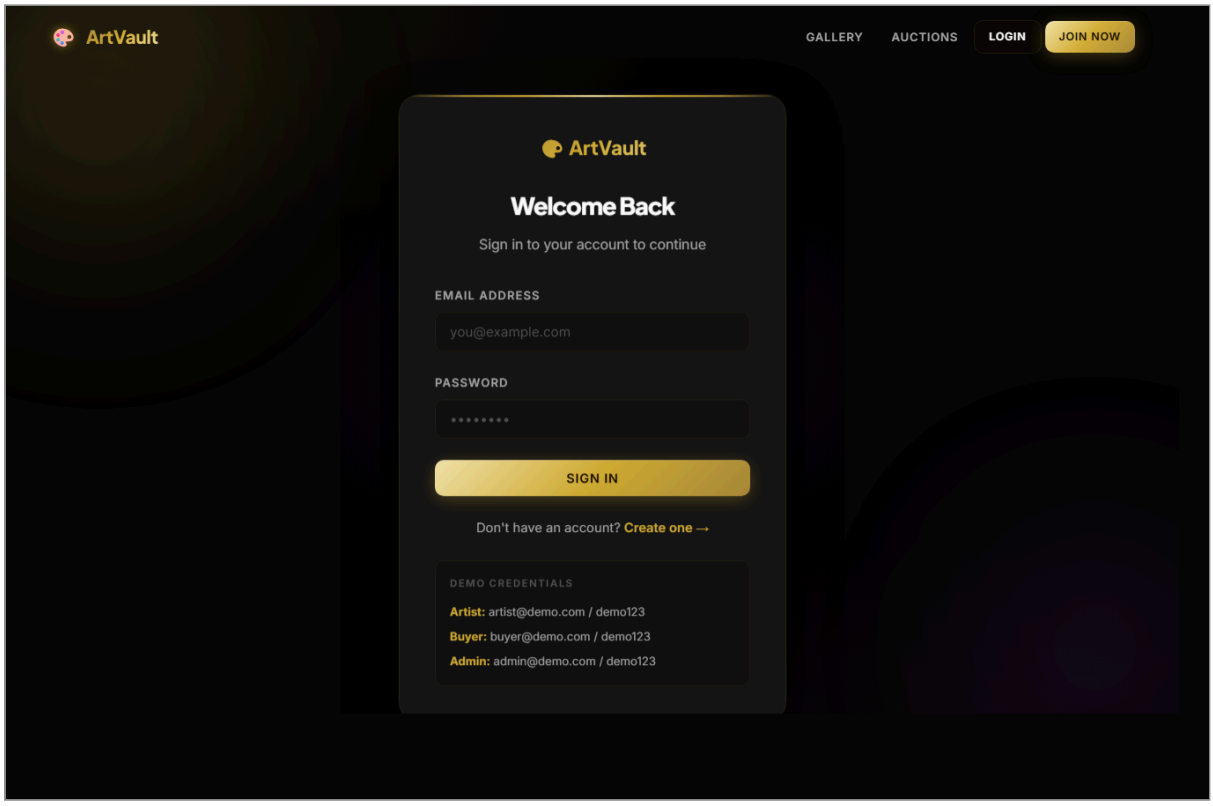


Figure 7.9: Protected Notifications Route



8. TESTING

Testing was performed by validating user flows, protected route behavior, API responses, data display, and frontend build quality. Public pages were checked in the browser and backend health was verified through the health API.

8.1 Test Cases

- Register User: Verify buyer or artist account creation with required fields.
- Login User: Verify valid users receive token and dashboard access.
- Browse Gallery: Verify artworks load and filters update results.
- View Artwork Details: Verify artwork information, artist, price, status, and image display.
- View Auctions: Verify active auctions and bid values are visible.
- Place Bid: Verify buyer can submit a bid higher than current highest bid.
- Protected Routes: Verify unauthenticated users are redirected to login.
- Admin Dashboard: Verify platform statistics and management tables load for admin role.
- Build and Lint: Verify frontend checks pass without lint errors.

8.2 Validation Result

- Frontend local server responded successfully at http://localhost:5173.
- Backend health endpoint responded successfully at http://localhost:5000/api/health.
- Artwork API returned populated artwork records.
- Auction API returned active auction records with artwork and bidder data.
- Screenshots were captured from the live local application.

9. FUTURE SCOPE

- Integrate a real payment gateway such as Razorpay or Stripe.
- Add advanced auction rules such as bid increments, reserve price, and auto-extension.
- Add email notifications for bid updates and payment confirmations.
- Add artist verification and artwork approval workflow.
- Add order shipment tracking and delivery partner integration.
- Add analytics reports for artists and administrators.
- Improve image storage using cloud services such as Cloudinary or AWS S3.

10. CONCLUSION

ArtVault successfully demonstrates a full-stack online art gallery and auction platform. The project combines a responsive React frontend, Express backend, MongoDB database, JWT authentication, role-based access, real-time auction updates, mock payment flow, and dashboards for different users. It provides a practical solution for artists and buyers and can be extended into a production-ready marketplace with payment, verification, and deployment enhancements.

11. REFERENCES

- React Documentation - https://react.dev/
- Vite Documentation - https://vite.dev/
- Express Documentation - https://expressjs.com/
- MongoDB Documentation - https://www.mongodb.com/docs/
- Mongoose Documentation - https://mongoosejs.com/
- Socket.IO Documentation - https://socket.io/docs/
- Redux Toolkit Documentation - https://redux-toolkit.js.org/